DATA STRUCTURES

ASSIGNMENT NO:2

1.Define an integer array. Declare an integer pointer and initialize it to pointer and initialize it to point to first element of array. use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

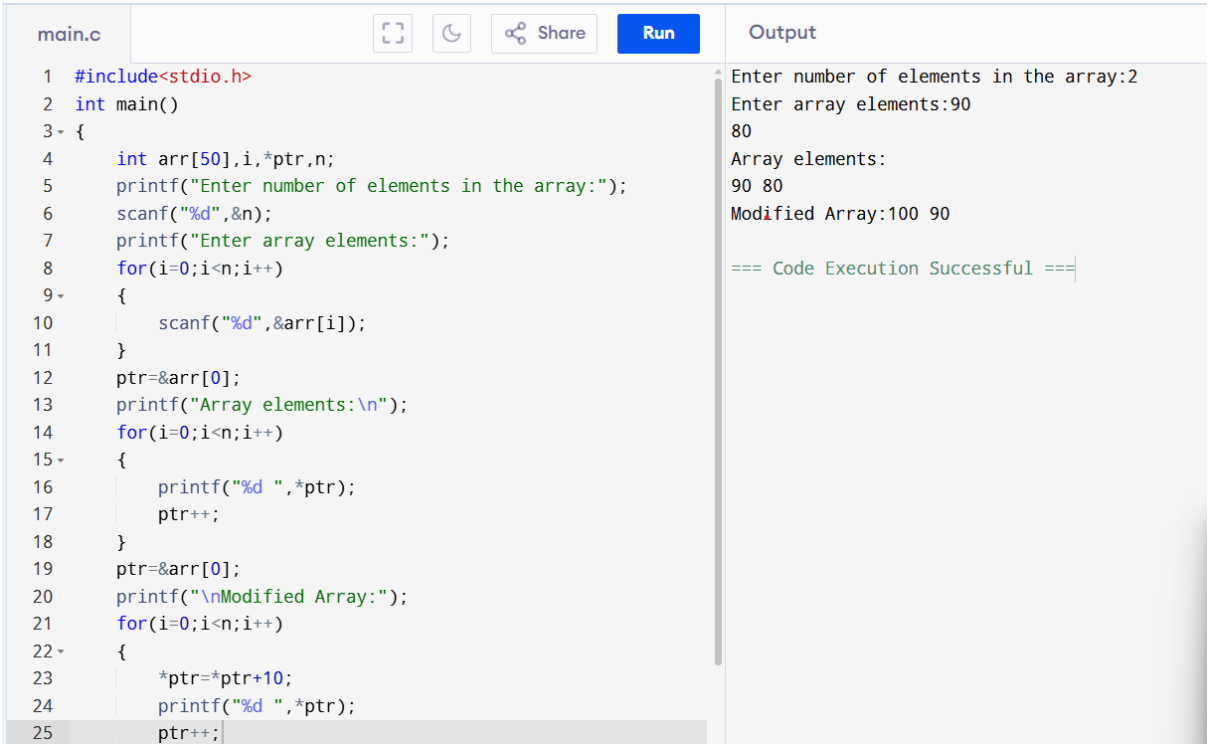
PROGRAM:

```
#include<stdio.h>
int main()
{
    int arr[50],i,*ptr,n;
    printf("Enter number of elements in the array:");
    scanf("%d",&n);
    printf("Enter array elements:");
    for(i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    ptr=&arr[0];
    printf("Array elements:\n");
    for(i=0;i<n;i++)
    {
        printf("%d ",*ptr);
        ptr++;
    }
    ptr=&arr[0];
    printf("\nModified Array:");
    for(i=0;i<n;i++)
    {
```

```
        *ptr=*ptr+10;
        printf("%d ",*ptr);
        ptr++;
    }

    return 0;
}
```

OUTPUT:



2.Calculate string length using pointers.

PROGRAM:

```
#include<stdio.h>
int main()
{
    char *ptr,str[25];
    int i,len=0;
    printf("Enter a string:");
```

```
    scanf("%[^\n]",str);
    ptr=str;
    for(i=0;str[i]!='\0';i++)
    {
        len++;
    }
    printf("Length of the string: %d",len);
    return 0;
}
```

OUTPUT:



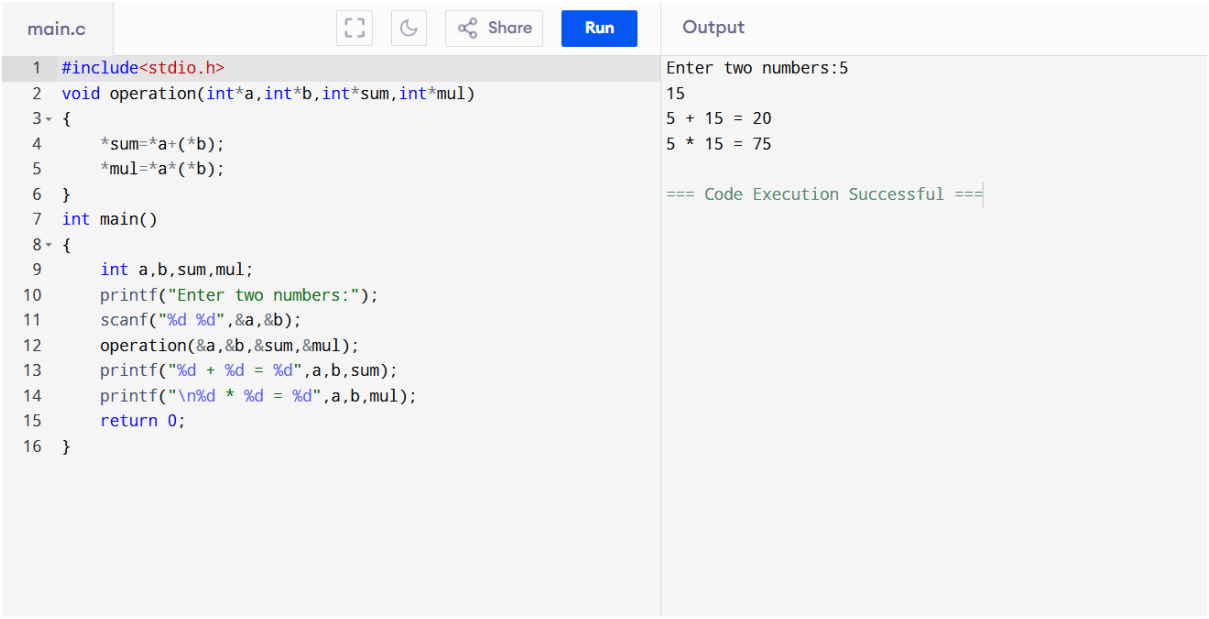
3.Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

PROGRAM:

```
#include<stdio.h>
void operation(int*a,int*b,int*sum,int*mul)
{
    *sum=*a+(*b);
    *mul=*a*(*b);
}
```

```
int main()
{
    int a,b,sum,mul;
    printf("Enter two numbers:");
    scanf("%d %d",&a,&b);
    operation(&a,&b,&sum,&mul);
    printf("%d + %d = %d",a,b,sum);
    printf("\n%d * %d = %d",a,b,mul);
    return 0;
}
```

OUTPUT:



4. Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access or modify members.

PROGRAM:

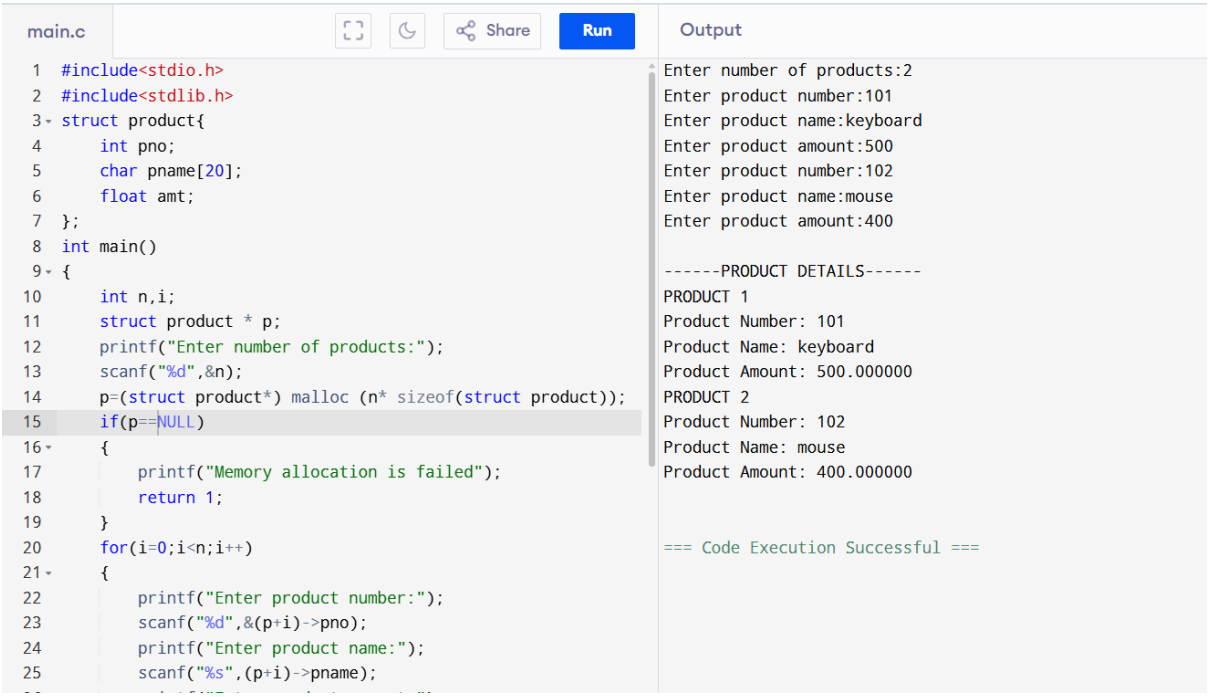
```
#include<stdio.h>
#include<stdlib.h>
struct product{
    int pno;
    char pname[20];
```

```
    float amt;
};
int main()
{
    int n,i;
    struct product * p;
    printf("Enter number of products:");
    scanf("%d",&n);
    p=(struct product*) malloc (n* sizeof(struct product));
    if(p==NULL)
    {
        printf("Memory allocation is failed");
        return 1;
    }
    for(i=0;i<n;i++)
    {
        printf("Enter product number:");
        scanf("%d",&(p+i)->pno);
        printf("Enter product name:");
        scanf("%s",(p+i)->pname);
        printf("Enter product amount:");
        scanf("%f",&(p+i)->amt);
    }
    printf("\n------PRODUCT DETAILS------\n");
    for(i=0;i<n;i++)
    {
        printf("PRODUCT %d\n",i+1);
        printf("Product Number: %d\n",(p+i)->pno);
        printf("Product Name: %s\n",(p+i)->pname);
```

```
        printf("Product Amount: %f\n",(p+i)->amt);
    }


    free(p);
    return 0;
}
```

OUTPUT:



main.c

```
#include<stdio.h>
#include<stdlib.h>
struct product{
    int pno;
    char pname[20];
    float amt;
};
int main()
{
    int n,i;
    struct product * p;
    printf("Enter number of products:");
    scanf("%d",&n);
    p=(struct product*) malloc (n* sizeof(struct product));
    if(p==NULL)
    {
        printf("Memory allocation is failed");
        return 1;
    }
    for(i=0;i<n;i++)
    {
        printf("Enter product number:");
        scanf("%d",&(p+i)->pno);
        printf("Enter product name:");
        scanf("%s",(p+i)->pname);
```

Output

```
Enter number of products:2
Enter product number:101
Enter product name:keyboard
Enter product amount:500
Enter product number:102
Enter product name:mouse
Enter product amount:400

------PRODUCT DETAILS------
PRODUCT 1
Product Number: 101
Product Name: keyboard
Product Amount: 500.000000
PRODUCT 2
Product Number: 102
Product Name: mouse
Product Amount: 400.000000


=== Code Execution Successful ===
```

5.Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

PROGRAM:

```
#include<stdio.h>

int sum(int a,int b)

{

    return a+b;

}

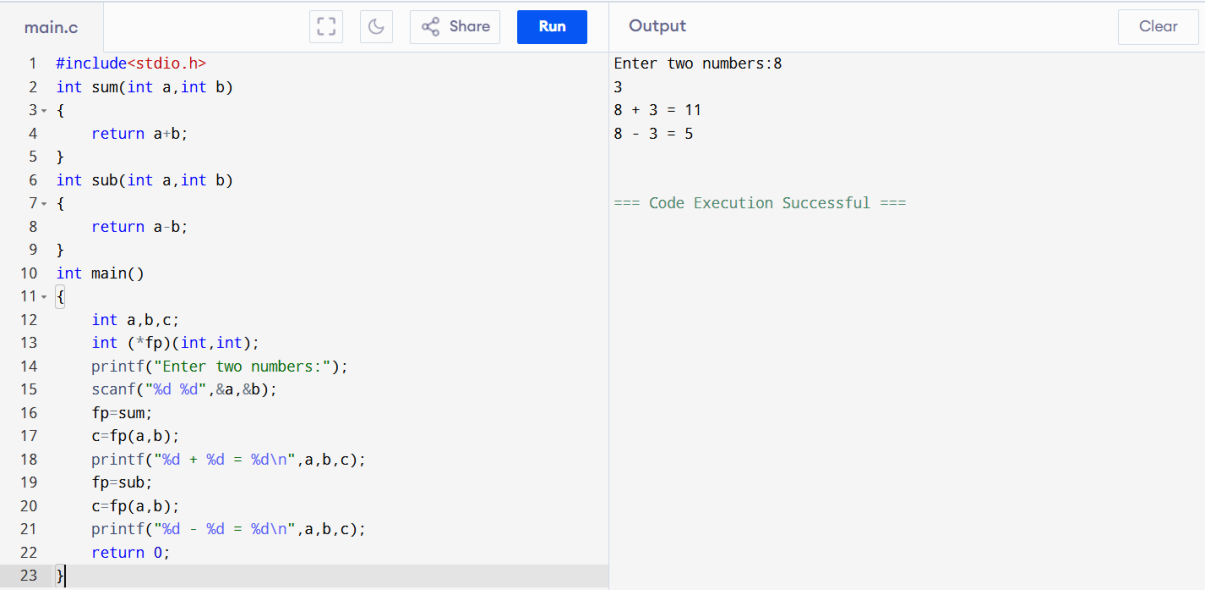
int sub(int a,int b)

{

    return a-b;
```

```
}
int main()
{
    int a,b,c;
    int (*fp)(int,int);
    printf("Enter two numbers:");
    scanf("%d %d",&a,&b);
    fp=sum;
    c=fp(a,b);
    printf("%d + %d = %d\n",a,b,c);
    fp=sub;
    c=fp(a,b);
    printf("%d - %d = %d\n",a,b,c);
    return 0;
}
```

OUTPUT:



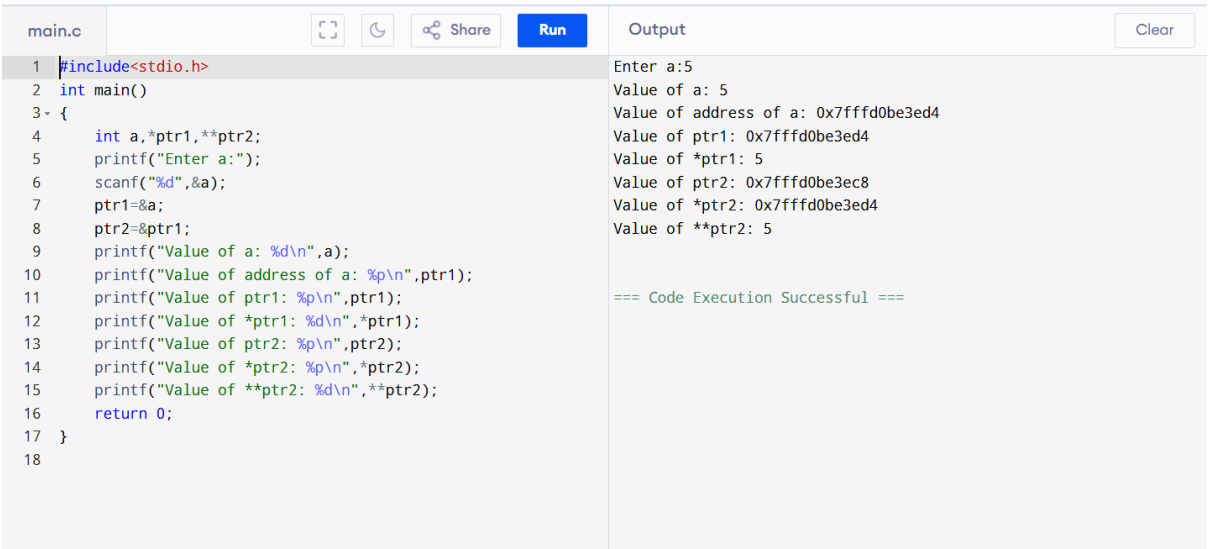
6.Declare an integer and a pointer to it and a double pointer. Print the values and addresses at each level.

PROGRAM:

```
#include<stdio.h>
```

```
int main()
{
    int a,*ptr1,**ptr2;
    printf("Enter a:");
    scanf("%d",&a);
    ptr1=&a;
    ptr2=&ptr1;
    printf("Value of a: %d\n",a);
    printf("Value of address of a: %p\n",ptr1);
    printf("Value of ptr1: %p\n",ptr1);
    printf("Value of *ptr1: %d\n",*ptr1);
    printf("Value of ptr2: %p\n",ptr2);
    printf("Value of *ptr2: %p\n",*ptr2);
    printf("Value of **ptr2: %d\n",**ptr2);
    return 0;
}
```

OUTPUT:



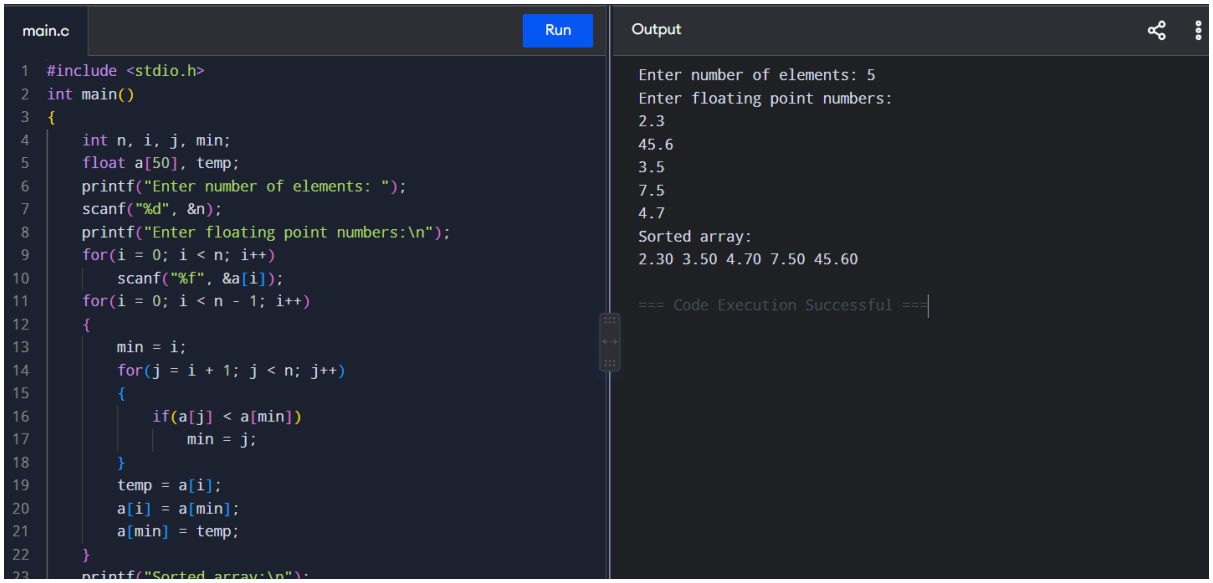
7. Sort an array of floating point numbers using selection sort.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
{
    int n, i, j, min;
    float a[50], temp;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter floating point numbers:\n");
    for(i = 0; i < n; i++)
        scanf("%f", &a[i]);
    for(i = 0; i < n - 1; i++)
    {
        min = i;
        for(j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%.2f ", a[i]);
    return 0;
}
```

OUTPUT:



8. Define a structure student with id, name and grade. Create an array of student structure and sort them based on different features.

PROGRAM:

```
#include <stdio.h>

#include <string.h>


struct student

{

    int id;

    char name[20];

    float grade;

};


int main()

{

    struct student s[50], temp;

    int n, i, j, choice;


    printf("Enter number of students: ");

    scanf("%d", &n);
```

```
printf("Enter student details:\n");

for(i = 0; i < n; i++)
{
    scanf("%d %s %f", &s[i].id, s[i].name, &s[i].grade);
}

printf("1. Sort by ID\n");
printf("2. Sort by Name\n");
printf("3. Sort by Grade\n");
printf("Enter choice: ");
scanf("%d", &choice);

for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(choice == 1 && s[i].id > s[j].id)
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
        else if(choice == 2 && strcmp(s[i].name, s[j].name) > 0)
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
```

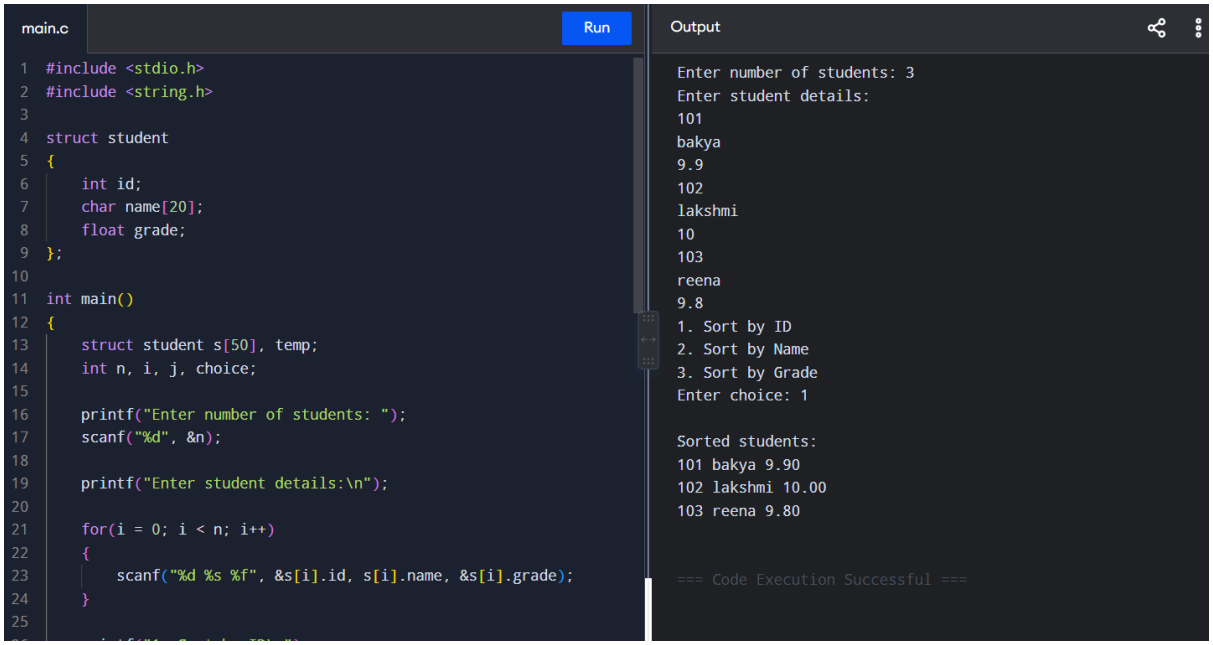
```
            }
            else if(choice == 3 && s[i].grade > s[j].grade)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nSorted students:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d %s %.2f\n",
               s[i].id, s[i].name, s[i].grade);
    }

    return 0;
}
```

OUTPUT:



9. Sort a set of negative numbers using radix sort.

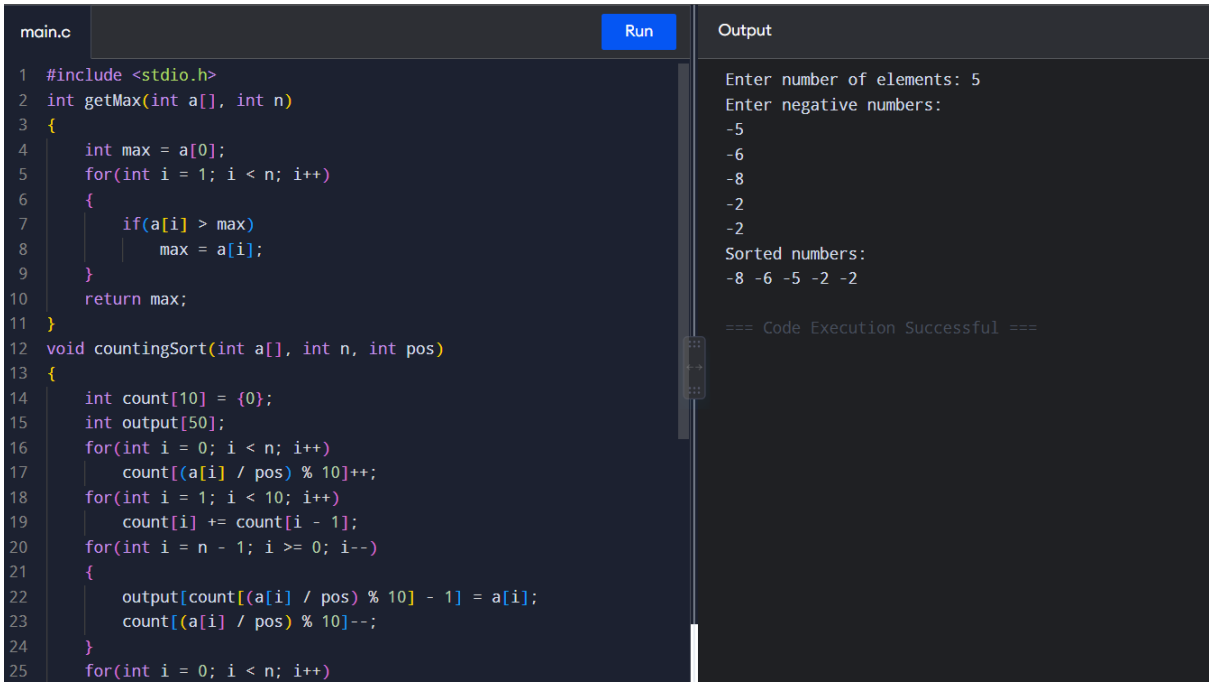
PROGRAM:

```
#include <stdio.h>
int getMax(int a[], int n)
{
    int max = a[0];
    for(int i = 1; i < n; i++)
    {
        if(a[i] > max)
            max = a[i];
    }
    return max;
}
void countingSort(int a[], int n, int pos)
{
    int count[10] = {0};
    int output[50];
    for(int i = 0; i < n; i++)
```

```
        count[(a[i] / pos) % 10]++;
    for(int i = 1; i < 10; i++)
        count[i] += count[i - 1];
    for(int i = n - 1; i >= 0; i--)
    {
        output[count[(a[i] / pos) % 10] - 1] = a[i];
        count[(a[i] / pos) % 10]--;
    }
    for(int i = 0; i < n; i++)
        a[i] = output[i];
}
void radixSort(int a[], int n)
{
    int max = getMax(a, n);
    for(int pos = 1; max / pos > 0; pos *= 10)
        countingSort(a, n, pos);
}
int main()
{
    int a[50], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter negative numbers:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for(int i = 0; i < n; i++)
        a[i] = -a[i];
    radixSort(a, n);
    printf("Sorted numbers:\n");
```

```
    for(int i = n - 1; i >= 0; i--)
        printf("%d ", -a[i]);
    return 0;
}
```

OUTPUT:



10. Write a c program to copy one array to another using pointers.

PROGRAM:

```
#include <stdio.h>
int main()
{
    int a[50], b[50], n, i;
    int *p, *q;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter array elements: ");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
    p = a;
    q = b;
    for(i = 0; i < n; i++)
    {
        *q = *p;
        p++;
        q++;
    }
    printf("Copied array: ");
    for(i = 0; i < n; i++)
        printf("%d ", b[i]);
    return 0;
}
```

OUTPUT:

